

Practical No.4-B Write a CUDA Program for the Matrix Multiplication using CUDA C

Name: Vedant Joshi

Roll No: 24

matrix_multiplication.cu -

```
#include <stdio.h>

#include <cuda_runtime.h>

#define N 512 // Matrix size N x N (you can change to 256 or 1024)

__global__ void matrixMul(float *a, float *b, float *c, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < n && col < n) {
        float sum = 0.0f;
        for (int k = 0; k < n; k++) {
            sum += a[row * n + k] * b[k * n + col];
        }
        c[row * n + col] = sum;
    }
}

int main() {a
    float *h_a, *h_b, *h_c; // host matrices
    float *d_a, *d_b, *d_c; // device matrices
    size_t size = N * N * sizeof(float);
    // Allocate host memory
    h_a = (float*)malloc(size);
    h_b = (float*)malloc(size);
    h_c = (float*)malloc(size);
    // Initialize matrices A = 1.0, B = 2.0
    for (int i = 0; i < N * N; i++) {
        h_a[i] = 1.0f;
        h_b[i] = 2.0f;
    }
}
```

```

// Allocate device memory
cudaMalloc(&d_a, size);
cudaMalloc(&d_b, size);
cudaMalloc(&d_c, size);

// Copy host to device
cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

// Launch kernel (16x16 threads per block is efficient)
dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (N + 15) / 16);
matrixMul<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, N);

// Copy result back
cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

// Print top-left 4x4 corner to verify
printf("Matrix Multiplication Result (Top-left 4x4):\n");
for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 4; j++) {
        printf("%.1f ", h_c[i * N + j]);
    }
    printf("\n");
}

// Expected value: every element should be  $N * 1.0 * 2.0 = 1024.0$ 
printf("\nExpected value for all elements = %.1f\n", (float)N * 2.0f);

// Free memory
free(h_a); free(h_b); free(h_c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);

printf("\nMatrix multiplication completed successfully on GPU!\n");
return 0;
}

```

Output:

vedant@LAPTOP-2AJF2JRG: nvcc -o matrix matrix.cu

matrix.cu

tmpxft_000001a0_00000000-7_matrix.cudafe1.cpp

vedant@LAPTOP-2AJF2JRG: matrix.exe

Matrix Multiplication Result (Top-left 4x4):

1024.0 1024.0 1024.0 1024.0

1024.0 1024.0 1024.0 1024.0

1024.0 1024.0 1024.0 1024.0

1024.0 1024.0 1024.0 1024.0

Expected value for all elements = 1024.0

Matrix multiplication completed successfully on GPU!